

Technical Report: OutputManager Class (Group-23)

1. Introduction

The OutputManager class is a unified logging utility which serves as the central component responsible for routing, filtering, and formatting log messages across the World/Agent simulation and related data analytics components. The class provides a lightweight and configurable interface that supports multiple log levels, optional timestamps and metadata, and routing of messages to the console, to a file, or to an in-memory buffer.

In a modular software design, output management is deliberately separated from business logic for several reasons. First, it allows the rest of the application to remain agnostic of where and how messages are delivered; components simply invoke the logging interface without depending on file paths, stream types, or formatting rules. Second, a single output module can enforce consistent formatting and filtering policies across all subsystems, which simplifies debugging and auditing. Third, separating output logic makes it easier to test the application by redirecting or capturing logs without altering production code. The OutputManager embodies this separation by encapsulating all concerns related to log level filtering, message formatting, and multi-target delivery within one cohesive unit.

2. Class Overview and Architecture

2.1 Class Structure

The OutputManager is defined in the namespace cse498 and consists of a single class accompanied by three supporting types: the LogLevel enum class, the OutputTarget enum class, and the LogContext struct. The class declares a default constructor and destructor, a set of configuration and query methods, and a core logging interface that is exposed both as a generic log method and as convenience methods for each log level.

2.2 Member Variables

The class maintains six private data members. The log filtering level is stored in a LogLevel member defaulted to Normal. Two boolean flags control whether timestamps and metadata (tag, agent identifier, tick) are included in formatted output. A bitmask of type unsigned int records which output targets are enabled: each value of the OutputTarget enum corresponds to a bit, so Console, File, and Buffer can be toggled

independently. The class holds an `std::ofstream` for file output and a `std::vector<std::string>` for the in-memory buffer. A mutable `std::mutex` is used to synchronize access to shared state from multiple threads.

2.3 Constructor and Destructor Behavior

The constructor initializes the target bitmask so that only the Console target is enabled by default; File and Buffer start disabled. No log file is opened in the constructor. The destructor acquires the mutex, checks whether the file stream is open, and closes it if so, ensuring that file resources are released during object destruction. This approach provides basic RAII-style cleanup for the file handle while keeping the rest of the object's lifecycle simple.

2.4 Access Modifiers

All client-facing operations are public: construction, destruction, configuration (`setLevel`, `enableTimestamps`, `enableMetadata`, `enableTarget`), file operations (`openLogFile`, `closeLogFile`), buffer access (`getBufferedLogs`, `clearBuffer`), and the logging methods (`log`, `logNormal`, `logVerbose`, `logDebug`). Implementation details and internal helpers are private. The private methods are documented as requiring the mutex to be held by the caller; they perform level checking, message formatting, and writing to the enabled targets without acquiring the lock themselves, avoiding deadlock and keeping locking policy centralized in the public methods.

3. Detailed Method Analysis

3.1 Configuration and Query Methods

The method `setLevel` accepts a `LogLevel` and updates the internal level under the mutex. The method `getLevel` returns the current level and is marked `[[nodiscard]]`; it also locks the mutex and returns a copy of the level. The methods `enableTimestamps` and `enableMetadata` each take a boolean and set the corresponding flag under the mutex. The method `enableTarget` takes an `OutputTarget` and a boolean; it maps the target to a bit via an internal helper, then either sets or clears that bit in the targets bitmask, again under the mutex.

The method `openLogFile` accepts a constant reference to a path string and returns a boolean. Under the mutex it closes any already-open file, clears the stream state, and opens the file with `std::ios::out | std::ios::trunc`. Success is determined by converting the stream to `bool`. On failure, the implementation re-enables the Console target so that output is not lost, then returns `false`; no exceptions are thrown. The method `closeLogFile` acquires the mutex and closes the file stream if it is open.

The method `getBufferedLogs` is `const` and returns `std::vector<std::string>`. It locks the mutex and returns a copy of the buffer, so callers receive a snapshot without holding the lock. The method `clearBuffer` acquires the mutex and clears the buffer vector.

3.2 Logging Methods and Internal Logic

The convenience methods `logNormal`, `logVerbose`, and `logDebug` each take a message string and an optional `LogContext` (defaulted to an empty context). They forward to the core log method with the appropriate `LogLevel`. The core log method takes a level, a message, and an optional context. It acquires the mutex once, then calls `ShouldLogUnlocked` to determine whether the message should be emitted given the current filter level. If not, it returns. Otherwise it calls `FormatMessageUnlocked` to produce a single formatted line and then `WriteLineUnlocked` to send that line to all enabled targets.

The private method `ShouldLogUnlocked` implements the filtering rules: `Silent` allows nothing; `Normal` allows only `Normal` messages; `Verbose` allows `Normal` and `Verbose`; `Debug` allows all levels. `FormatMessageUnlocked` builds the line in an `std::ostringstream`. If timestamps are enabled, it obtains the current system time, converts it to a broken-down time structure using a platform-specific function (`localtime_s` on Windows, `localtime_r` elsewhere), and appends a bracketed timestamp in the form `[YYYY-MM-DD HH:MM:SS]`. It then appends the level string in brackets, and if metadata is enabled it appends the tag (if non-empty), and optionally `agent=<id>` and `tick=<value>` when the context provides non-negative values. Finally it appends the message and returns the string.

The method `WriteLineUnlocked` checks each target in turn. If `Console` is enabled, it writes the line to `std::cout` followed by a newline. If `File` is enabled, it writes to the file stream if open, otherwise falls back to `std::cout`. If `Buffer` is enabled, it pushes the line onto the buffer. The static method `LevelToString` maps each `LogLevel` to a string constant for inclusion in the formatted output. The method `IsTargetEnabledUnlocked` returns whether the given target's bit is set in the targets bitmask.

4. Design Principles and Patterns

The class exhibits strong encapsulation: all state is private, and the only way to change behavior or produce output is through the public API. Configuration is explicit and granular (level, timestamps, metadata, per-target enablement), which keeps the interface clear and testable.

The Single Responsibility Principle is respected in that the class has one well-defined responsibility: accept log messages with an optional context, filter them by level, format

them consistently, and deliver them to the configured targets. It does not interpret domain logic, perform I/O beyond logging, or manage other resources.

The design can be viewed as a combination of patterns. It acts as a facade over multiple output mechanisms (console, file, buffer), presenting a single log interface. It also functions as a logger-style abstraction: callers use level-specific methods or the generic log method without caring about formatting or destination. The use of a bitmask for targets and the internal Unlocked helpers suggest a small internal layer that assumes external synchronization, which keeps the locking policy in one place and avoids nested locking.

5. File I/O and Stream Handling

File output is handled through a single `std::ofstream` member. The file is opened in `openLogFile` with truncation semantics, so each call to `openLogFile` replaces the previous file content. All writes to the file occur from `WriteLineUnlocked`, which is only called while holding the mutex, so concurrent log calls are serialized and file writes are thread-safe with respect to other `OutputManager` operations.

The error handling strategy is defensive and exception-free from the caller's perspective. If `openLogFile` fails, it returns false and re-enables the Console target so that subsequent log messages still appear on the console. When File is enabled but the stream is not open, `WriteLineUnlocked` writes to `std::cout` instead, ensuring that no log is dropped silently. The destructor closes the file stream if open, providing basic resource cleanup (RAII for the file handle at destruction time). The stream object itself is not managed by a separate RAII wrapper; ownership and lifetime are tied to the `OutputManager` instance.

6. Testing and Validation

The test suite in `OutputManagerTests.cpp` is built with Catch2 and is organized into sections covering log level filtering, output targets, formatting options, file handling, buffer APIs, and a basic thread-safety check.

Log level tests construct an `OutputManager` with only the Buffer target enabled and verify that at Silent no messages appear in the buffer; at Normal only Normal messages appear; at Verbose both Normal and Verbose appear; and at Debug all three levels appear. Target tests verify that when only Buffer is enabled, messages show up in `getBufferedLogs`; when only Console is enabled, output is captured using a `CoutRedirect` helper that redirects `std::cout` to an `std::ostringstream` for assertion; and when only File is enabled, messages are written to a temporary file that is read back and checked. Multiple-target tests confirm that the same message appears in both the buffer and the file when both targets are enabled.

Formatting tests assert that with timestamps disabled, the buffer content does not contain date-like or time-like patterns, and that with timestamps enabled, the output contains bracketed segments and colons. Metadata tests verify that when metadata is disabled, the tag does not appear, and when enabled, the tag and optional agent and tick values appear as expected. File tests check that a valid path leads to a successful open and written content, that an invalid path causes `openLogFile` to return false without crashing, and that when the file open fails, the buffer target still receives messages. Buffer tests verify the count and content of buffered logs and that `clearBuffer` empties the buffer and that subsequent logs are stored correctly. A multithreaded test spawns several threads that each log many messages to the buffer and asserts that the total count matches and that each line contains the expected content, providing a basic check that the mutex protects shared state.

The `TestHelpers.hpp` component provides a temporary log path, a function to remove the temp file, and the `CoutRedirect` RAII class that redirects `std::cout` to an in-memory stream and restores it on destruction, enabling tests to capture and assert on console output without affecting the global cout state permanently.

7. Integration in the Project

Other modules in the project use the `OutputManager` by instantiating it (or receiving it via dependency injection), configuring the desired level and targets, and then calling `logNormal`, `logVerbose`, `logDebug`, or `log` with an optional `LogContext`. The `OutputManagerDemo.cpp` program illustrates this: it sets the level to `Debug`, enables timestamps and metadata, enables `Console` and `Buffer`, optionally opens a file and enables the `File` target, then logs messages with different context values and finally prints the buffered lines. In a full simulation, world or agent components would similarly obtain an `OutputManager` reference and log events with tags and optional agent and tick information.

Centralized output handling yields consistent formatting and filtering across all subsystems, a single place to adjust log level or targets (e.g., for production versus debugging), and the ability to capture logs in a buffer for replay or analysis without changing the code that emits the logs.